Python Namespaces, Scope & Decorators - Beginner Notes

1. Namespaces

What is a Namespace?
A namespace is a container that holds a mapping of names (identifiers) to objects. It ensures that names are unique and can be used without conflict.

Types of Namespaces:

1. **Built-in Namespace** - Contains built-in functions like `print()`, `len()`, `range()`
2. **Global Namespace** - Contains names defined at the module level
3. **Local Namespace** - Contains names defined inside a function

Built-in namespace (always available)

print(len([1, 2, 3])) # len is in built-in namespace

Global namespace

x = 10 # x is in global namespace

def my_function():
# Local namespace
y = 20 # y is in local namespace of my_function
print(x) # Can access global x
print(y) # Can access local y

my_function()

print(y) # Error! y is not accessible outside function

Key Points:

- Namespaces prevent naming conflicts
- Each namespace is created at different times and has different lifetimes
- Namespaces are implemented as Python dictionaries

---

2. Scope & LEGB Rule

What is Scope?
Scope defines the region in code where a variable is accessible.

LEGB Rule:
Python follows the LEGB rule to resolve variable names:

- **L**ocal - Inside current function
- **E**nclosing - Inside enclosing functions (nested)
- **G**lobal - At module level
- **B**uilt-in - Python's built-in names

Example demonstrating LEGB

x = "Global" # Global scope

def outer():
x = "Enclosing" # Enclosing scope

```
def inner():
    x = "Local"  # Local scope
    print("Inner:", x)  # Prints: Inner: Local

inner()
print("Outer:", x)  # Prints: Outer: Enclosing
```

outer()
print("Global:", x) # Prints: Global: Global

How Python Searches:

1. First checks **Local** scope
2. Then checks **Enclosing** scope (if exists)
3. Then checks **Global** scope
4. Finally checks **Built-in** scope
5. If not found, raises `NameError`

---

3. Local & Global Scope

Local Scope

Variables defined inside a function are local.

def my_function():
local_var = "I am local"
print(local_var) # Accessible inside function

```
my_function() # Output: I am local
```

print(local_var) # Error: NameError

Function parameters are also local:

```
def greet(name): # name is local
message = "Hello" # message is local
print(message, name)

greet("Alice") # Output: Hello Alice
```

print(name) # Error: NameError

Global Scope

Variables defined at module level are global.

```
global_var = "I am global"

def my_function():
print(global_var) # Can read global variable

my_function() # Output: I am global
print(global_var) # Output: I am global
```

Modifying Global Variables:

```
counter = 0 # Global variable

def increment():
global counter # Declare as global to modify
counter += 1
print("Counter:", counter)

increment() # Output: Counter: 1
increment() # Output: Counter: 2
print(counter) # Output: 2
```

Without `global` keyword (creates local variable):

```
x = 10 # Global

def my_function():
x = 20 # Creates NEW local variable
print("Inside:", x) # Output: Inside: 20

my_function()
print("Outside:", x) # Output: Outside: 10 (unchanged)
```

---

4. Built-in Scope

Built-in scope contains Python's pre-defined names.

These are in built-in namespace

```
print(len([1, 2, 3])) # Output: 3
print(max([1, 5, 3])) # Output: 5
print(sum([1, 2, 3])) # Output: 6
print(abs(-10)) # Output: 10
print(round(3.7)) # Output: 4
```

Checking built-in names:

```
import builtins
print(dir(builtins)) # Shows all built-in names
```

Built-in scope is searched last:

Overriding built-in names (NOT recommended)

```
len = 100 # Overrides built-in len function
print(len) # Output: 100
```

Now len() function won't work

print(len([1, 2, 3])) # Error: 'int' object is not callable

Better to use different name

```
my_len = 100
print(len([1, 2, 3])) # Works fine: 3
```

Example showing LEGB with built-in:

If variable not found in L, E, G, Python checks B (built-in)

def my_function():
# 'print' is not in Local, Enclosing, or Global
# Python finds it in Built-in scope
print("Hello") # Uses built-in print function

my_function() # Output: Hello

---

5. Enclosing Scope

Enclosing scope exists in nested functions.

def outer():
x = "Enclosing" # Enclosing scope for inner()

```
def inner():
    print(x)  # Accesses x from enclosing scope

inner()
```

outer() # Output: Enclosing

Multiple levels of nesting:

def outer():
x = "Outer"

```
def middle():
    x = "Middle"

    def inner():
        print(x)  # Accesses x from middle (nearest enclosing)

    inner()

middle()
```

outer() # Output: Middle

Using nonlocal keyword:

def outer():
count = 0 # Enclosing scope variable

```
def inner():
    nonlocal count  # Modify enclosing scope variable
    count += 1
    print("Count:", count)

inner()  # Output: Count: 1
inner()  # Output: Count: 2
print("Final count:", count)  # Output: Final count: 2
```

outer()

Without nonlocal (creates local variable):

def outer():
count = 0

```
def inner():
    count = 1  # Creates NEW local variable
    print("Inner count:", count)

inner()  # Output: Inner count: 1
print("Outer count:", count)  # Output: Outer count: 0 (unchanged)
```

outer()

Practical Example - Closure:

def multiplier(n):
def multiply(x):
return x * n # n is from enclosing scope
return multiply

times_2 = multiplier(2)
times_5 = multiplier(5)

print(times_2(10)) # Output: 20
print(times_5(10)) # Output: 50

---

6. Decorators

What is a Decorator?
A decorator is a function that modifies the behavior of another function without changing its code.

Basic Decorator:

```
def my_decorator(func):
def wrapper():
print("Before function call")
func()
print("After function call")
return wrapper

@my_decorator
def say_hello():
print("Hello!")

say_hello()
```

Output:

Before function call

Hello!

After function call

How Decorators Work:

This decorator syntax:

```
@my_decorator
def say_hello():
print("Hello!")
```

Is equivalent to:

```
def say_hello():
print("Hello!")
say_hello = my_decorator(say_hello)
```

Decorator with Function Arguments:

```
def my_decorator(func):
def wrapper(*args, **kwargs):
```

```
print("Before function call")
result = func(*args, **kwargs)
print("After function call")
return result
return wrapper

@my_decorator
def add(a, b):
print(f"Adding {a} + {b}")
return a + b

result = add(5, 10)
print("Result:", result)
```

Output:

Before function call

Adding 5 + 10

After function call

Result: 15

Practical Example - Timer Decorator:

```
import time

def timer(func):
def wrapper(*args, **kwargs):
start = time.time()
result = func(*args, **kwargs)
end = time.time()
print(f"{func.name} took {end - start:.4f} seconds")
return result
return wrapper

@timer
def slow_function():
time.sleep(2)
print("Function completed")

slow_function()
```

Output:

Function completed

slow_function took 2.0001 seconds

Practical Example - Login Required Decorator:

```
def login_required(func):
def wrapper(user):
if user.get("logged_in"):
return func(user)
else:
print("Please login first!")
return wrapper

@login_required
def view_profile(user):
print(f"Welcome {user['name']}!")

user1 = {"name": "Alice", "logged_in": True}
user2 = {"name": "Bob", "logged_in": False}

view_profile(user1) # Output: Welcome Alice!
view_profile(user2) # Output: Please login first!
```

Multiple Decorators:

```
def uppercase_decorator(func):
def wrapper():
result = func()
return result.upper()
return wrapper

def exclamation_decorator(func):
def wrapper():
result = func()
return result + "!!!"
return wrapper

@exclamation_decorator
@uppercase_decorator
def greet():
return "hello"

print(greet()) # Output: HELLO!!!
```

Decorator with Arguments:

def repeat(times):
def decorator(func):
def wrapper(*args, **kwargs):
for _ in range(times):
result = func(*args, **kwargs)
return result
return wrapper
return decorator

@repeat(times=3)
def say_hello():
print("Hello!")

say_hello()

Output:

Hello!

Hello!

Hello!

Class as Decorator:

class CountCalls:
def **init**(self, func):
self.func = func
self.count = 0

```
def __call__(self, *args, **kwargs):
    self.count += 1
    print(f"Call {self.count} of {self.func.__name__}")
    return self.func(*args, **kwargs)
```

@CountCalls
def greet():
print("Hello!")

greet() # Output: Call 1 of greet \n Hello!
greet() # Output: Call 2 of greet \n Hello!
greet() # Output: Call 3 of greet \n Hello!

Preserving Function Metadata:

from functools import wraps

```
def my_decorator(func):
@wraps(func) # Preserves original function's metadata
def wrapper(*args, **kwargs):
print("Before")
result = func(*args, **kwargs)
print("After")
return result
return wrapper

@my_decorator
def greet(name):
"""Greets a person"""
print(f"Hello, {name}!")

greet("Alice")
print(greet.name) # Output: greet (without @wraps, would be 'wrapper')
print(greet.doc) # Output: Greets a person
```

Quick Reference

Concept	Description
Namespace	Container mapping names to objects
LEGB Rule	Local → Enclosing → Global → Built-in
Local Scope	Variables inside function
Global Scope	Variables at module level
Enclosing Scope	Variables in outer function (nested)
Built-in Scope	Python's pre-defined names
global keyword	Modify global variable from function
nonlocal keyword	Modify enclosing variable from nested function
Decorator	Function that modifies another function
@decorator syntax	Clean way to apply decorator

Table 1: Key Concepts Summary

Code	Purpose
`global x`	Modify global variable x
`nonlocal x`	Modify enclosing variable x
`@decorator`	Apply decorator to function
`@wraps(func)`	Preserve function metadata
`*args, **kwargs`	Accept any arguments in decorator

Table 2: Common Patterns

Scope Resolution Order:

1. Check Local scope first
2. Then check Enclosing scope (if nested)
3. Then check Global scope
4. Finally check Built-in scope
5. Raise NameError if not found

When to Use:

- **global:** When modifying module-level variables from functions
- **nonlocal:** When modifying outer function variables from nested functions
- **Decorators:** To add functionality without modifying original code (logging, timing, authentication, caching)